Fichier réponse

TP1-Jour 1:

TP1-1:

Combien de temps à duré le fun ? Où voit-on les logs de chaque étapes?

Le run à duré 20 sec. Pour trouvé les logs il faut cliqué sur le push présent dans Actions puis cliqué sur le run pour avoir tout l'affichage.

TP1-4

Quel test à échoué ? À quelle ligne ? Qu'est ce qui a permis de le trouver rapidement ?

Le test test_is_valid_title_accepts_normal_title() à la ligne 13. Grâce à l'affichage des logs on peut déterminer simplement d'où vient l'erreur.

TP1-7

Pourquoi attend-on que la CI soit verte avant de fusionner ?

On attend afin que tout les modifications soient prise en compte

TP2-Jour 2:

TP2-1:

Quelle différence entre un Linter (flake8) et un formateur (black) ? Pourquoi séparer Lint et test en deux jobs ?

Flake8 détecte les erreurs dans le code (import non utilisé, variable non définie) alors que black défini un style dans la manière d'écrire les fichiers et détecte donc si tout les fichiers suivent cette manière sinon ils les reformatent.

Séparer Lint et test en deux jobs permet de séparer le métier de chacun, cela permet donc d'identifier quelle jobs provoque une erreur.

TP2-4:

Pourquoi tester sur plusieurs versions de python ? Que gagne-t-on avec le cache ? Un artefact, c'est quoi ?

On test sur plusieurs versions de python pour savoir si le code ne casse pas sur les différentes versions, on veut être sur qu'il est utilisable sur les versions différentes sur lesquelles il peut être utilisé.
Le cache permet d'aller plus vite et pour cela on n'a pas besoin de retélécharger les dépendances.
Un artefact est un fichier produit par un job et sauvegardé à la fin du job, pour pouvoir le télécharger ou le réutiliser dans une étape suivante

TP2-5:

Pourquoi interdire de pousser directement sur la main ?

On interdit de pousser directement sur la main pour éviter de push un code qui ne passe pas les tests donc qui risque de tout casser.

TP2-8:

Pourquoi git ne peut-il pas résoudre ce conflit seul ? Qu'est ce qui a vérifié le code après la résolution ?

Git ne peux pas résoudre le conflit seul car afin que tout soit résolu il faut soit suprimmer du code soit modifier, donc git ne sait pas ce qu'il faut changer.
Après la résolution le code est add commit puis push. Dans Github la rule que j'ai mis en place vérifie que tout est passé.

TP2-9:

Pourquoi GitHub masque-t-il le secret ? Que se passe-t-il si on le commit dans le dépôt ? Secret ou variable: comment choisir ?

GitHub masque le secret pour ne pas rendre publique des token par exemple. Tout les commit publié sur GitHub sont visibles par tous et pour toujours. Si on le commit dans le dépôt il est masqué avec des ****. Les variables sont des données peu sensibles que l'on va souvent utiliser et que l'on a pas besoin de cacher alors que le secret sont des données que l'on va utiliser plus souvent mais qui sont beaucoup plus sensible comme des token c'est pour cela qu'il faut les cacher.

TP2-11

pipeline_A: mauvaise indentation
pipeline_B: manque de uses pour avoir accès au fichier des dossiers
pipeline_C: manque des quote pour les python-version

Pour remedier à tout ça il faut bien relire et utiliser des rules.

<u>TP3-jour 3:</u>

TP3-1:

Taille de l'image docker de tasks-api: 49.2MB

Étapes CACHED:

=> CACHED [2/6] WORKDIR /app
0.0s
=> CACHED [3/6] COPY requirements.txt .
0.0s
=> CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt

Si on copie le code avant le requirements.txt le pop Install va se rééxécuter à chaque fois car le code change tout le temps. Pour éviter cela on copy d'abords le requirements puis on fait le pip install et comme celui ci ne change pas le pic Install ne se réexecute pas.

Quelle est la différence entre une image et un conteneur ? A quoi sert le dockerignore ?

Un conteneur se base sur une image pour se construire. L'image sert de base/recette pour chaque conteneur basé sur celle-ci. Elle pose les bases pour les container et on build une image a partir d'un dockerfile que l'on définit au préalable.

Le .dockerignore sert à ne pas envoyer certains fichiers ou dossiers dans l'image lors du docker build, donc ils ne se retrouvent pas dans les container

TP3-2:

Que gagne-t-on à décrire le lancement dans un fichier plutôt que dans une longue commande ?

On gagne de la lisibilité et du contrôle sur ce que l'on lance, on à moins de chance de faire des fautes de frappes et cela permet de relancer rapidement un build plutôt que de retaper la longue commande à chaque fois.

TP3-3

Pourquoi `build` attend-il `test` ? Pourquoi éviter le tag `latest` en production ?

Le test intall des dépendances et build se base sur ce que test fait.
Il faut éviter latest car des tag précis montre clairement l'importance du changement apporté.

TP3-5

À quoi sert un test de fumée ? Pourquoi demander une approbation humaine avant la production ?

Ce sont des test rapide en surface. Pourquoi qu'un humain donne son accord pour le déploiement.

TP3-8

Car si l'application est piraté le hacker a toi les droit dans le conteneur

TP4-jour 4:

TP4-1:

C'est quoi un pod ? Un deployment ? Pourquoi un `namespace` ?

Un pod est un conteneur effemère. Un deployment est un maintien d'image par kubernetes. Un namespace sert à isoler et organiser les ressources d'un cluster Kubernetes

TP4-4:

On sépare les deux dépôts parce que le code et le déploiement ont des cycles de vie et des droits d'accès différents.

TP4-7:

C'est l'option d'Argo CD qui corrige automatiquement les dérive entre le cluster et Git.
Git est le chef d'orchestre donc il suffit juste de faire un git revers pour pouvoir revenir en arrière.

TP4-10

1. Le git taf est push, la ci se déclenche
2. La ci construit l'image puis push
3. La ci push encore le tag de l'image dans le dépôt
4. Argo cd dans le cluster pull le spot et détecte lecart avec lestât réel
5. Il applique les manifeste puis kubelet tire l'image et démarre lepod

La CI pousse tandis que Argo cd et kubelet tire l'image

En classique, la pipeline pousse directement dans le cluster avec Kubectl apply ou helm upgrade.
Elle doit donc détenir les accès au cluster.
En GitOps, c'est le cluster qui tire. La CI n'a aucun accès au cluster, ce qui est plus sûr.